

Reporte de Actividad — Simulador de Estacionamiento

Práctica 2

Pablo Fernando Ruiz Perez 379207

Jose Carlos Gallegos Mariscal

3 de abril de 2026

1. Introducción

En esta práctica se desarrolló un Simulador de Estacionamiento aplicando el paradigma de Programación Orientada en python.

Esta actividad fue dividida en 3 sesiones:

- Sesión 1: Modelo orientado a objetos y comportamiento básico.
- Sesión 2: Polimorfismo, subtipos y extensiones controladas.
- Sesión 3: Interfaz web con Flask.

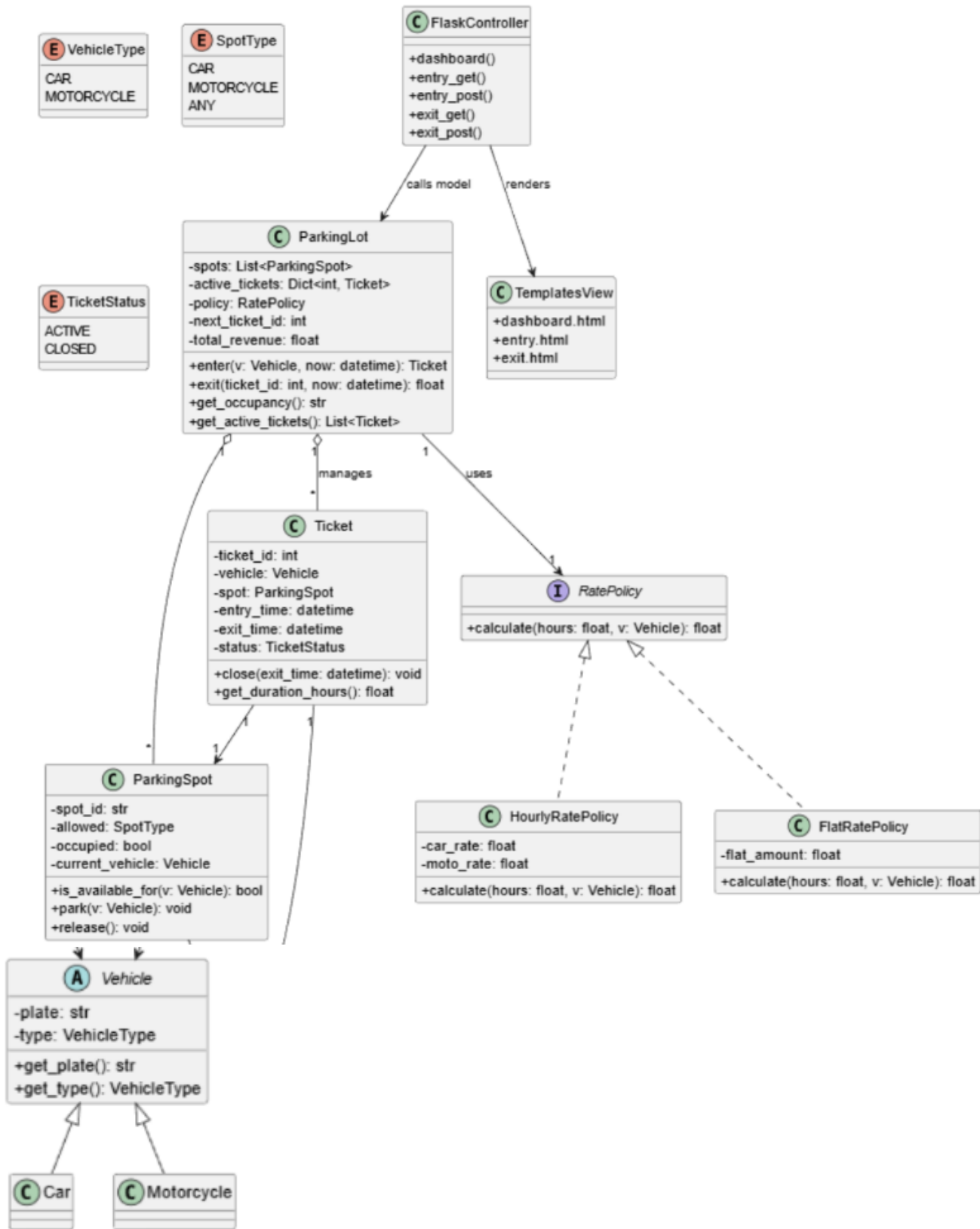
2.Objetivo

Diseñar e implementar un sistema simple con las siguientes características; modelo, encapsulación, abstracción, herencia, composición comportamiento, polimorfismo, subtipos y una interfaz web con Flask.

2.1. Lista de Clases y Responsabilidades

- **Vehicle**: Representa la abstracción de un vehículo con sus placas y tipo.
- **ParkingSpot**: Administra un lugar específico, controlando su estado de ocupación y compatibilidad.
- **Ticket**: Registra la transacción de entrada, vinculando el vehículo con el lugar y calculando el tiempo de estancia.
- **ParkingLot**: Clase principal que coordina la lógica de negocio, administra la colección de lugares y procesa ingresos/egresos.
- **RatePolicy**: Define la interfaz para el cálculo de costos, permitiendo separar las políticas de cobro del resto del sistema.

2.2. Diagrama UML



3. Evidencia de Conceptos POO

[cite_start]A continuación se presentan fragmentos de código (snippets) que demuestran la implementación de los pilares de POO requeridos[cite: 379]:

3.1. Encapsulación

Los atributos están protegidos y las validaciones ocurren dentro de los métodos para mantener los invariantes del sistema.

```
# Ejemplo en ParkingSpot
def park(self, v: Vehicle) -> None:
    if not self.is_available_for(v): # Validación de invariante
        raise ValueError("El spot no es compatible o ya está ocupado.")
    self._occupied = True # Atributo privado protegido
    self._current_vehicle = v
```

3.2. Abstracción

Se utiliza una interfaz para que la lógica de cobro no esté ligada directamente al sistema.

```
# Ejemplo en ParkingSpot
class RatePolicy(Protocol):
    def calculate(self, hours: float, v: Vehicle) -> float:
```

3.3. Composición

La clase principal ParkingLot está compuesta por una colección (lista) de ParkingSpot y administra el ciclo de vida de los Ticket mediante un diccionario.

```
class ParkingLot:
    def __init__(self, spots: List[ParkingSpot], policy: RatePolicy):
        self._spots = spots
        self._active_tickets: Dict[int, Ticket] = {}
```

3.4. Herencia y subtipos

Se implementaron las clases específicas Car y Motorcycle que heredan de la clase base abstracta Vehicle, definiendo su tipo en el constructor.

```
class Vehicle(ABC):
    # ... métodos base e inicialización ...

class Car(Vehicle):
    def __init__(self, plate: str):
        super().__init__(plate, VehicleType.CAR)

class Motorcycle(Vehicle):
    def __init__(self, plate: str):
        super().__init__(plate, VehicleType.MOTORCYCLE)
```

3.5. Polimorfismo

El método `calculate` de `HourlyRatePolicy` evalúa polimórficamente el objeto `Vehicle` que recibe, aplicando una tarifa distinta según el tipo de vehículo en tiempo de ejecución, sin que `ParkingLot` conozca el tipo específico.

```
def calculate(self, hours: float, v: Vehicle) -> float:
    if v.get_type() == VehicleType.CAR:
        return hours * self._car_rate
    elif v.get_type() == VehicleType.MOTORCYCLE:
        return hours * self._moto_rate
    return hours * 15.0 # Tarifa base
```

4. MVC con Flask

La arquitectura se dividió siguiendo el patrón Modelo-Vista-Controlador de forma simplificada:

Model (Modelo): Se reutilizaron las clases del dominio intactas desarrolladas en las sesiones previas (`ParkingLot`, `Ticket`, `Vehicle`, `ParkingSpot`, `RatePolicy`). Estas agrupan toda la lógica de negocio.

View (Vista): Se construyeron plantillas HTML mediante el motor Jinja2 (como `dashboard.html` y `entry.html`). Estas se encargan de renderizar la información de ocupación y los formularios.

Controller (Controlador): El archivo `app.py` funciona como enrutador. Captura las solicitudes HTTP (GET/POST), interactúa extrayendo o modificando el estado del modelo `ParkingLot` y finalmente invoca a las vistas usando `render_template()`.

4.1. Captura de rutas y pantallas

Entrada

Práctica 2: Pro...PP_PL2_Galleg...PP_PL2_Galleg...Tarea de Inves...Parking L...x+

127.0.0.1:5000/entry

Presentación Mi pro...MIT App InventorEl texto del párrafo...Documento sin titul...

DashboardRegistrar EntradaRegistrar Salida

Registrar Entrada de Vehículo

Placas:
2131as

Tipo de Vehículo:
Motocicleta (MOTORCYCLE) ▾

Dar de Entrada

Buscar

17°C
Prac. despejado

ESP

Salida

Práctica 2: Programación Ori...xPP_PL2_Gallegos_40032.pdf - Go...PP_PL2_Gallegos_40032.pdf - Go...Tarea de Investigación: Fundame...Parking Lot Simulatorx+

127.0.0.1:5000/exit

Presentación Mi pro...MIT App InventorEl texto del párrafo...Documento sin titul...

DashboardRegistrar EntradaRegistrar Salida

Registrar Salida

ID del Ticket:
1

Horas transcurridas (Simulación):
3

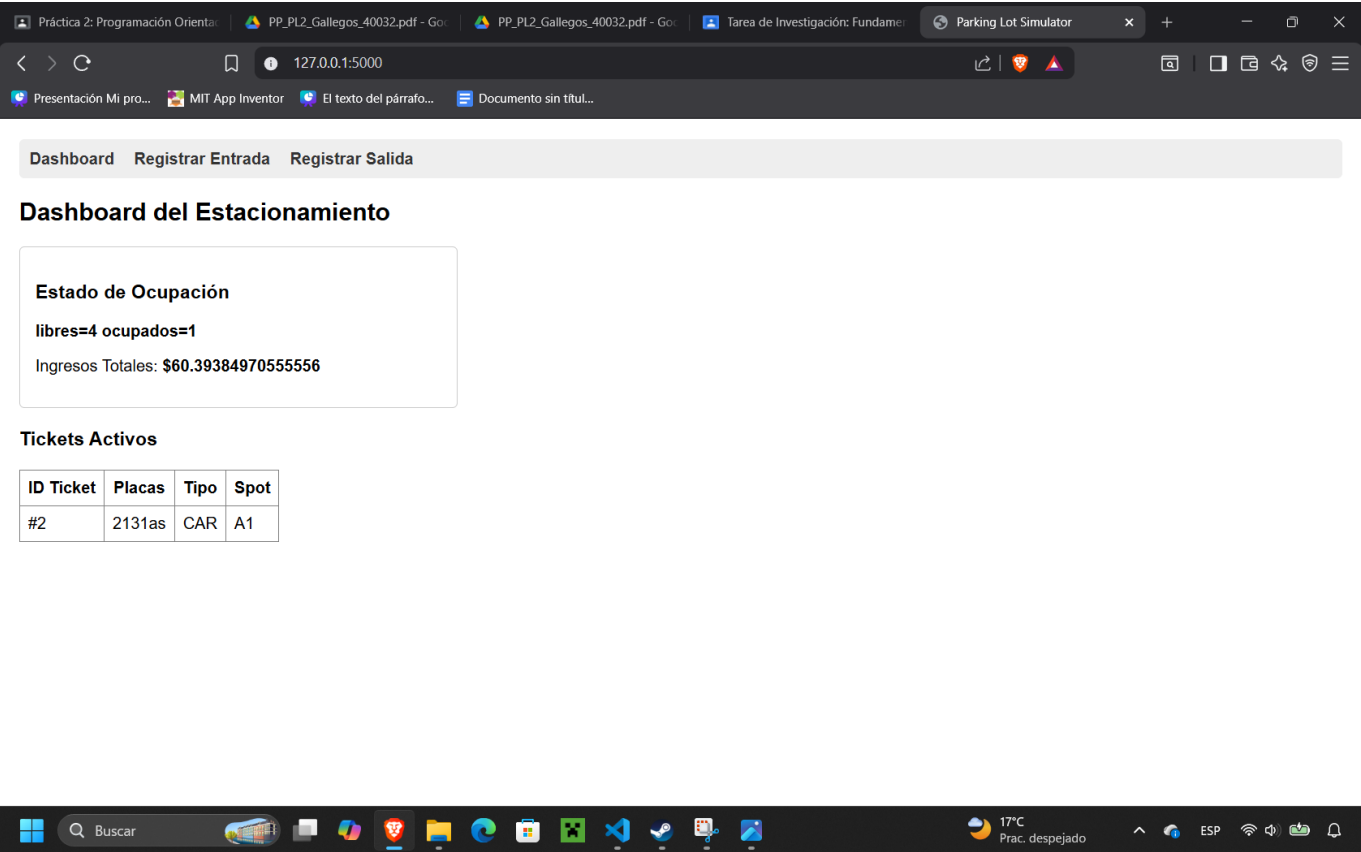
Procesar Salida y Cobrar

Buscar

17°C
Prac. despejado

ESP

Dashboard



5. Flujos

Flujo 1:

```
--- Simulador de Estacionamiento ---
1. Registrar entrada
2. Registrar salida
3. Ver ocupación
4. Ver tickets activos
5. Salir
Elige una opción: 1
Placas: 341es21d
Tipo (CAR/MOTORCYCLE): CAR
Entrada exitosa -> Ticket #1, spot=A1

--- Simulador de Estacionamiento ---
1. Registrar entrada
2. Registrar salida
3. Ver ocupación
4. Ver tickets activos
5. Salir
Elige una opción: 2
ID del ticket: 1
Horas transcurridas (simulación): 3
Salida: ticket=1 -> tiempo=3.0h costo $60.00 spot liberado=A1
```

Flujo2:

Dashboard Registrar Entrada Registrar Salida

Registrar Entrada de Vehículo

Placas:

AS1E23

Tipo de Vehículo:

Automóvil (CAR) ▼

Dar de Entrada

Dashboard Registrar Entrada Registrar Salida

Entrada exitosa. Ticket #1 asignado al spot A1

Dashboard del Estacionamiento

Estado de Ocupación

libres=4 ocupados=1

Ingresos Totales: \$0.0

Tickets Activos

ID Ticket	Placas	Tipo	Spot
#1	AS1E23	CAR	A1

Dashboard Registrar Entrada Registrar Salida

Registrar Salida

ID del Ticket:

Horas transcurridas (Simulación):

Procesar Salida y Cobrar

Dashboard **Registrar Entrada** **Registrar Salida**

Salida procesada. Costo a cobrar: \$82.55

Dashboard del Estacionamiento

Estado de Ocupación

libres=5 ocupados=0

Ingresos Totales: \$82.55354763333332

Tickets Activos

No hay vehículos en el estacionamiento.

7 Preguntas Guía

1. ¿Qué clase concentra la responsabilidad de asignar spots y por qué?

Parking_Lot, ya que es la única que puede tener acceso a toda la información necesaria para administrar el estacionamiento.

2. ¿Qué invariantes protege tu modelo? (menciona al menos 2).

El parking spot se garantiza que no puede ser ocupado por más de un vehículo con el atributo privado `__occupied`.

Los tickets solo pueden ser cerrados una vez, ya que estos cuentan con un método `closed` que cambie su estatus.

3. ¿Dónde se aplica polimorfismo y qué ventaja aporta en tu diseño?

Se aplica principalmente en la interfaz de cobro (`RatePolicy`) y en los tipos de vehículos (`Car`, `Motorcycle`). En la implementación, el método `calculate` de `HourlyRatePolicy` evalúa polimórficamente al objeto `Vehicle` que recibe para decidir la tarifa correcta. Por ello el código no es tan confuso y no necesita una gran cantidad de `if else` para evaluar esto.

4. ¿Qué parte del sistema pertenece a Model, View y Controller en tu Flask?

Model pertenece a todas las clases creadas dentro de models, como vehicle, parkingSpot y ticket.

View pertenece a todos los archivos html y css que están dentro de template, estos ayudan a que sea visible la interfaz de la web.

Controller flask pertenece al archivo app.py que proporciona toda la administración de flask implementado las clases creadas de models.

5. Si mañana cambian las tarifas, ¿qué clase(es) tocarías y por qué?

Desde hourlyRatePolicy ya que esta clase se encarga de las políticas de cobro, como el cobro por auto, motocicleta y asigna una tarifa a cada uno.

8. Conclusiones

El desarrollo de este simulador permitió integrar progresivamente los pilares fundamentales de la Programación Orientada a Objetos. Mediante la encapsulación y la composición, se garantizó la integridad de los datos, evitando colisiones en la ocupación de espacios. Posteriormente, la implementación de herencia y polimorfismo demostró cómo el código puede extenderse para soportar distintas reglas de negocio sin necesidad de reescribir la lógica central del estacionamiento. Finalmente, la adopción del patrón MVC con el framework Flask facilitó la creación de una interfaz web funcional que respeta y reutiliza por completo el modelo de dominio subyacente.